



Reporte documentación

Garcia Gonzalez Luis Angel
Estrada Ortiz Jesús Rigoberto
Velez Gutierrez Alma Citlali
Zuno Segura Nestor
Programación de Sistemas Avanzados

Introducción

El presente reporte documenta el diseño, desarrollo e implementación de una infraestructura distribuida de alto rendimiento para la generación y renderizado del conjunto fractal de mandelbrot. Esta actividad consiste en construir un sistema capaz de dividir una carga de trabajo matemáticamente intensiva entre nodos de procesamiento.

Marco Teórico

El proyecto se sustenta en tres pilares fundamentales:

1. **Calculo Fractal:** Consiste en la iteración de la función comenzando con $Z_{n+1} = Z_n^2 + C$. Un punto C pertenece al conjunto si, al iterar la función empezando con $Z_0 = 0$, el valor absoluto de Z_n , no se desvía. La visualización de este conjunto requiere procesar millones de píxeles de forma independiente, lo que convierte a este algoritmo en un buen ejemplo para aplicar la paralelización.
2. **Topología de Red Hub-and-Spoke:** Se tiene el modelo de red donde hay un nodo central que actuara como nuestro hub y coordinador, y otros múltiples nudos periféricos que serán los workers y se comunicaran solo con el centro, aislando la red y centralizando el control del estado.
3. **VPN con WireGuard:** Para tener la comunicación segura y estructurada entre todos los nodos físicos, se implementaron túneles que encapsulan el tráfico, permitiendo que todos los contenedores interactúen como si estuvieran en una red de área local aislada.

La implementación usa lenguajes de programación con manejo seguro de memoria usando Rust y plataformas de manejo de contenedores Docker, herramientas para construir sistemas robustos y escalables.

Objetivo

Diseñar e implementar un sistema distribuido con contenedores dockers que funcione a través de túneles de una VPN privada para conectar y que permita ejecutar el renderizado del algoritmo de Mandelbrot de forma conjunta y asíncrona con los dispositivos conectados utilizando el lenguaje Rust.

Objetivos Particulares

- Implementar un nodo Coordinador que es encargado de segmentar la imagen y distribuir las tareas bajo la demanda de trabajo.
- Crear nodos de trabajo sin estado que calculen los algoritmos iterativos del fractal y retornen matrices de píxeles.
- Configurar una red virtual utilizando WireGuard para la comunicación e ininterrumpida entre el coordinador y los ejecutores.
- Aislar los servicios mediante Docker Compose para asegurar una escalabilidad horizontal fluida y la limitación estricta de recursos de hardware.

Desarrollo

Para llevar a cabo este proyecto, nuestro equipo adoptó un enfoque de desarrollo modular abordando tanto la configuración de red y contenerización, basado en una arquitectura de microservicios y una red VPN, Hub-and-Spoke. El objetivo principal fue construir un sistema distribuido capaz de renderizar imágenes del fractal de Mandelbrot en resoluciones extremas (hasta 4K), dividiendo la carga entre múltiples nodos.

A continuación, detallamos la metodología, los módulos desarrollados y las herramientas utilizadas en cada fase del proyecto:

1. Diseño de la infraestructura VPN

Para poder interconectar el nodo coordinador con los nodos trabajadores, se creó una topología Hub-and-Spoke utilizando WireGuard. Durante la creación, se identificaron que el subsistema de windows para linux presenta limitaciones para gestionar interfaces de red virtuales de forma nativa. Por ello, se optó por una configuración híbrida: el Hub de WireGuard se configuró y ejecutó de manera nativa en el Host físico asignando la IP 10.0.0.1.

Para permitir que los contenedores Docker recibieran las peticiones de los workers externos, fue necesario diseñar una regla de firewall en el host. Mediante una consola de PowerShell con los privilegios de administrador, se habilita el tráfico entrante en el puerto TCP 3005, con la sentencia "New-NetFirewallRule -DisplayName "Permitir API Gateway Mandelbrot" -Direction Inbound -LocalPort 3005 -Protocol TCP -Action Allow". Por otra parte, los workers en ubuntu se configuraron como clientes apuntando al endpoint público del Host, creando un túnel de cifrado.

2. Algoritmo de Mandelbrot en Rust

Todo el corazón del proyecto fue desarrollado en Rust ya que fue un requisito del proyecto, por su alta gestión de la memoria y rendimiento computacional ya que es necesario para calcular y renderizar miles de pixeles para un fractal así pudiendo también manejar múltiples hilos.

El cálculo del Mandelbrot es altamente demandante a nivel de procesador. Por lo tanto, el desarrollo del mismo en el worker requiere implementar matemática estricta optimizada en Rust para evitar latencias innecesarias.

La metodología que aplicamos para desarrollar el algoritmo es el siguiente:

1. Mapeo de coordenadas:

Se definieron los límites de un plano matemático utilizando números flotantes y doubles para no perder detalle visual en el renderizado final.

Se establecieron los ejes reales e imaginarios :

- Reales : $x_{\min} = -2.0$, $x_{\max} = 1.0$
- Imaginarios: $y_{\min} = -1.5$, $y_{\max} = 1.5$

2. Cálculo por filas:

El Hub asigna trabajo fila por fila y el worker calcula la coordenada imaginaria fija para esa fila. Después se inicializa un bucle que recorre cada pixel X de la imagen de la izquierda a la derecha que es el ancho total.

3. La ecuación de Mandelbrot:

Para cada pixel X que ahora es un punto C en el plano complejo con parte real e imaginario, se aplica la sucesión matemática vista en el marco teórico todo esto aplicado en el código de Rust mediante un while:

- Se inician variables temporales $z_{\text{real}} = 0.0$ y $z_{\text{imaginario}} = 0.0$.
- En cada una de las iteraciones se calculan los nuevos valores y se verifica la condición de escape si la magnitud del punto supera 2, es decir, $z_{\text{real}}^2 + z_{\text{imaginario}}^2 > 4.0$, el punto "escapa" hacia el infinito y detenemos el bucle.
- Si no escapa se continúa hasta llegar al límite máximo que se definió en el sistema.

4. Retorno de datos:

El algoritmo guarda el conteo de iteraciones de cada pixel en un vector unidimensional en memoria, Este vector representa una sola linea horizontal de la imagen, la cual se comprime y se envia enseguida por la red.

3. Implementación y Funcionalidad de los Módulos de Código (Rust)

Para lograr un sistema concurrente y libre de condiciones de carrera el código central se dividió en microservicios independientes gestionados por medio un workspace de Cargo. A continuación, se explica el funcionamiento algorítmico y lógico de cada uno:

3.1 Librería Compartida (shared):

Es el núcleo de la comunicación. Implementa las estructuras de datos universales del clúster: Task, TaskResult y LogMessage. Se hizo uso de la librería serde con los traits Serialize y Deserialize para que las estructuras de Rust se traduzcan automáticamente al formato de JSON al transmitirse por la red, garantizando la compatibilidad entre los contenedores. En TaskResult se integró la variable latency_ms para recopilar la telemetría del tiempo de procesamiento.

```
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct Task {
    pub task_id: u32,
    pub row: u32,
    pub width: u32,
    pub height: u32,
    pub max_iter: u32,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct TaskResult {
    pub worker_id: String,
    pub row: u32,
    pub data: Vec<u32>,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
```

```
pub struct LogMessage {
    pub source: String,
    pub level: String,
    pub message: String,
    pub timestamp: u64,
}
```

3.2 Coordinador Central (hub/[main.rs](#)):

En este módulo se utiliza un framework web asincrono Axum. El corazón del coordinador es su estructura AppState, que almacena el progreso global. Dado que múltiples workers solicitan y entregan tareas simultáneamente, se implementaron medidas de seguridad de memoria:

- Control de concurrencia: Las filas pendientes y completas utilizan AtomicU32, permitiendo que se incrementen de forma segura sin bloqueos. El lienzo de la imagen se envolvió en un Mutex para que solo un worker escriba sus resultados a la vez en la memoria RAM.
- Algoritmo de ensamblado: En el endpoint POST /result, al detectar que las filas completadas igualan al total, se ejecuta un doble ciclo for que itera sobre la matriz de resultados. Dependiendo de las iteraciones que tomo cada pixel, se le asigna un color RGB, poniendolo de negro si pertenece al conjunto, y guardando finalmente el buffer como mandelbrot.png

```
use axum::{
    routing::{get, post},
    Json, Router,
    extract::State,
};
*use shared::{Task, TaskResult};
use std::sync::{Arc, Mutex};
use std::sync::atomic::{AtomicU32, Ordering};
use image::{ImageBuffer, Rgb};
use serde::Serialize; // <-- Añadido para serializar el progreso

struct AppState {
    next_row: AtomicU32,
    completed_rows: AtomicU32,
    total_rows: u32,
    width: u32,
```

```

        image_data: Mutex<Vec<Vec<u32>>>>,
    }

    // <-- NUEVA ESTRUCTURA AGREGADA
    #[derive(Serialize)]
    struct Progress {
        completed: u32,
        total: u32,
    }

    #[tokio::main]
    async fn main() {
        let width = 3840;
        let height = 2160;

        let state = Arc::new(AppState {
            next_row: AtomicU32::new(0),
            completed_rows: AtomicU32::new(0),
            total_rows: height,
            width,
            image_data: Mutex::new(vec![vec![0; width as usize]; height as
usize]),
        });

        let app = Router::new()
            .route("/task", get(get_task))
            .route("/result", post(submit_result))
            .route("/progress", get(get_progress)) // <-- NUEVA RUTA AGREGADA
            .with_state(state);

        println!("Coordinador iniciado en 0.0.0.0:3000");

        axum::Server::bind(&"0.0.0.0:3000".parse().unwrap())
            .serve(app.into_make_service())
            .await
            .unwrap();
    }

    async fn get_task(State(state): State<Arc<AppState>>) -> Json<Task> {
        let row = state.next_row.fetch_add(1, Ordering::SeqCst);
    }

```

```

        if row < state.total_rows {
            Json(Task {
                task_id: row,
                row,
                width: state.width,
                height: state.total_rows,
                max_iter: 5000,
            })
        } else {
            Json(Task {
                task_id: 0,
                row: 9999,
                width: 0,
                height: 0,
                max_iter: 0,
            })
        }
    }
}

async fn submit_result(
    State(state): State<Arc<AppState>>,
    Json(payload): Json<TaskResult>,
) -> &'static str {

    let mut img_data = state.image_data.lock().unwrap();
    img_data[payload.row as usize] = payload.data;

    let completadas = state.completed_rows.fetch_add(1, Ordering::SeqCst)
+ 1;

    // Modificamos el mensaje para incluir la latencia
    println!(
        "Recibida fila {} de {} (por: {} | Latencia: {} ms)",
        payload.row,
        state.total_rows,
        payload.worker_id,
        payload.latency_ms // <--- IMPRIMIMOS LA LATENCIA AQUÍ
    );
}

```



```

if completadas == state.total_rows {
    println!("¡Cálculo finalizado! Generando mandelbrot.png...");

    let mut img = ImageBuffer::new(state.width, state.total_rows);

    for (y, row_data) in img_data.iter().enumerate() {
        for (x, &iter) in row_data.iter().enumerate() {

            let pixel_color = if iter == 5000 {
                Rgb([0, 0, 0])
            } else {
                let color = (iter % 256) as u8;
                Rgb([color, color, 255])
            };

            img.put_pixel(x as u32, y as u32, pixel_color);
        }
    }

    img.save("/output/mandelbrot.png").unwrap();
    println!("¡mandelbrot.png guardado!");
}

"OK"
}

// <-- NUEVA FUNCIÓN AGREGADA
async fn get_progress(State(state): State<Arc<AppState>>) ->
Json<Progress> {
    Json(Progress {
        completed: state.completed_rows.load(Ordering::SeqCst),
        total: state.total_rows,
    })
}

```

3.3 Ejecutores / Workes (worker/[main.rs](#))

Los nodos de los workers son nodos sin estado. Su ciclo de vida es un loop infinito que hace peticiones GET al Hub.

- Algoritmo de cálculo: Recibe las dimensiones y la fila específica. Mapea la coordenada real de la imagen al plano complejo matemático. Por cada pixel en su fila, aplica la iteración en un bucle while que se detiene si la magnitud escapa a 4.0 o se llega a 5000 iteraciones.
- Telemetría: Antes de llamar a `calcular_fila`, el código instala un reloj. Al finalizar el cálculo algorítmico, lee el tiempo transcurrido, convirtiéndolo a milisegundos y agregándole a la carga útil HTTP POST para que el Hub registre el esfuerzo computacional de ese nodo en específico.

```
use shared::{Task, TaskResult};
use std::env;
use tokio::time::{sleep, Duration};

fn calcular_fila(task: &Task) -> Vec<u32> {
    let mut fila_pixels = Vec::with_capacity(task.width as usize);

    let x_min = -2.0;
    let x_max = 1.0;
    let y_min = -1.5;
    let y_max = 1.5;

    let y_coord = y_min + (task.row as f64 / task.height as f64) * (y_max - y_min);

    for x in 0..task.width {
        let x_coord = x_min + (x as f64 / task.width as f64) * (x_max - x_min);

        let mut z_r = 0.0;
        let mut z_i = 0.0;
        let mut iter = 0;

        while z_r * z_r + z_i * z_i <= 4.0 && iter < task.max_iter {
            let temp = z_r * z_r - z_i * z_i + x_coord;
            z_i = 2.0 * z_r * z_i + y_coord;
```

```

        z_r = temp;
        iter += 1;
    }

    fila_pixels.push(iter);
}

fila_pixels
}

#[tokio::main]
async fn main() {
    let coordinator_url =
        env::var("COORDINATOR_URL").unwrap_or_else(|_|
"http://localhost:3000".to_string());

    let worker_id = format!("Worker-{}", std::process::id());

    let client = reqwest::Client::new();

    println!("{}", listo para calcular Mandelbrot...", worker_id);

    loop {
        if let Ok(res) = client
            .get(format!("{}/task", coordinator_url))
            .send()
            .await
        {
            if let Ok(task) = res.json::<<Task>().await {
                if task.row == 9999 {
                    println!("{}", No hay más trabajo. Esperando...",
worker_id);

                    sleep(Duration::from_secs(5)).await;
                    continue;
                }

                println!("{}", Calculando fila {}...", worker_id,
task.row);

                let pixels = calcular_fila(&task);

```

```

        let result = TaskResult {
            task_id: task.task_id,
            worker_id: worker_id.clone(),
            row: task.row,
            data: pixels,
        };

        let _ = client
            .post(format!("{}/result", coordinator_url))
            .json(&result)
            .send()
            .await;
    }
} else {
    sleep(Duration::from_secs(2)).await;
}
}
}

```

3.4 API Gateway ([api_gateway/main.rs](#)):

Funciona como un proxy inverso. Su función es exponer el puerto 3005 a la VPN y recibir todas las peticiones externas. Usando un cliente reqwest configurado con un límite de tiempo de 30 segundos para evitar bloqueos por latencia de red, este Gateway toma la petición del worker y la retransmite internamente al contenedor del Hub en el puerto 3000. Hace un retorno de la respuesta del Hub al worker, dando una capa de abstracción.

```

use axum::{
    routing::{get, post},
    Json, Router,
    extract::State,
    http::StatusCode,
    response::IntoResponse,
};

use shared::{Task, TaskResult, LogMessage};
use std::sync::Arc;
use std::net::SocketAddr;
use std::time::Duration;

```

```

// Estado compartido del Gateway
struct GatewayState {
    client: reqwest::Client,
    hub_url: String,
    logger_url: String,
}

#[tokio::main]
async fn main() {
    // Configuramos un cliente HTTP con tiempo de espera para evitar
    bloqueos

    let client = reqwest::Client::builder()
        .timeout(Duration::from_secs(30))
        .build()
        .unwrap();

    let state = Arc::new(GatewayState {
        client,
        hub_url: "http://coordinator:3000".to_string(), // Nombre del
servicio en Docker
        logger_url: "http://logger:4000/log".to_string(),
    });

    let app = Router::new()
        // Los trabajadores externos llamarán a estos puntos
        .route("/task", get(proxy_get_task))
        .route("/result", post(proxy_submit_result))
        .with_state(state);

    // Escuchamos en 0.0.0.0 para ser visibles desde la VPN (10.0.0.1)
    let addr = SocketAddr::from([0, 0, 0, 0], 3005);
    println!("--- API Gateway ---");
    println!("Escuchando en http://{", addr);
    println!("Redirigiendo a Hub en: http://coordinator:3000");

    axum::Server::bind(&addr)
        .serve(app.into_make_service())
        .await
        .unwrap();
}

```

```

}

// Lógica de Proxy para obtener tareas
async fn proxy_get_task(
    State(state): State<Arc<GatewayState>>
) -> impl IntoResponse {
    let url = format!("{}", state.hub_url);

    let res = state.client
        .get(&url)
        .send()
        .await;

    match res {
        Ok(response) => {
            if let Ok(task) = response.json::<Task>().await {
                (StatusCode::OK, Json(task)).into_response()
            } else {
                (StatusCode::INTERNAL_SERVER_ERROR, "Error al deserializar
tarea del Hub").into_response()
            }
        }
        Err(_) => (StatusCode::SERVICE_UNAVAILABLE, "El Coordinador (Hub)
no responde").into_response(),
    }
}

// Lógica de Proxy para enviar resultados
async fn proxy_submit_result(
    State(state): State<Arc<GatewayState>>,
    Json(payload): Json<TaskResult>,
) -> impl IntoResponse {
    let url = format!("{}", state.hub_url);

    let res = state.client
        .post(&url)
        .json(&payload)
        .send()
        .await;

```

```

match res {
    Ok(_) => (StatusCode::OK, "Resultado enviado al
Hub").into_response(),
    Err(_) => (StatusCode::SERVICE_UNAVAILABLE, "No se pudo entregar
el resultado al Hub").into_response(),
}
}

```

3.5 Monitor y Logger Centralizado (monitor/[main.rs](#) y logger/[main.rs](#))

El sistema cuenta con observabilidad. El monitor funciona como un proceso en segundo plano que, cada 5 segundos mediante un sleep asíncronico, solicita el endpoint GET /progress del Hub. Con los datos recabados, calcula una regla de tres para obtener el porcentaje de renderizado y envía este reporte a la red.

El logger es un servidor secundario escuchando por el puerto 4000. Recibe los paquetes JSON desde el monitor y los muestra en consola, así para que no se pierdan los mensajes de estado y permitiendo rastrear la evolución del fractal sin saturar la consola principal del coordinador.

Monitor:

```

use std::time::Duration;
use shared::LogMessage;
use serde::{Deserialize, Serialize};

#[derive(Deserialize)]
struct Progress {
    completed: u32,
    total: u32,
}

#[tokio::main]
async fn main() {
    let client = reqwest::Client::new();
    let hub_url = "http://coordinator:3000/progress";
    let logger_url = "http://logger:4000/log";

    println!("Monitor iniciado. Rastreando progreso del Mandelbrot...");

    loop {

```

```

        // 1. Intentamos obtener el progreso desde el Hub
        if let Ok(response) = client.get(hub_url).send().await {
            if let Ok(stats) = response.json::<<Progress>>().await {
                let porcentaje = (stats.completed as f32 / stats.total as
f32) * 100.0;

                println!("Progreso actual: {:.2}% ({} de {} filas)",
                    porcentaje, stats.completed, stats.total);

                // 2. Reportamos el progreso al Logger para centralizar la
info

                let log = shared::LogMessage {
                    source: "monitor".to_string(),
                    level: "info".to_string(),
                    message: format!("Progreso del renderizado: {:.2}%",
porcentaje),
                    timestamp: 0, // Aquí podrías usar tiempo real
                };

                let _ = client.post(logger_url).json(&log).send().await;

                if stats.completed == stats.total {
                    println!(";Monitor detectó que el renderizado ha
finalizado!");
                    break;
                }
            }
        }

        // Esperamos 5 segundos antes de la siguiente consulta
        tokio::time::sleep(Duration::from_secs(5)).await;
    }
}

```


Logger:

```
use axum::{routing::post, Json, Router};
use std::net::SocketAddr;
use shared::LogMessage; // Importamos la estructura de tu librería compartida

#[tokio::main]
async fn main() {
    // Definimos la ruta para recibir los logs del Hub, Monitor y Gateway
    let app = Router::new()
        .route("/log", post(handle_log));

    // Puerto 4000 como acordamos para evitar conflictos con el Hub (3000)
    let addr = SocketAddr::from([0, 0, 0, 0], 4000);
    println!("Logger iniciado y escuchando en http://{}", addr);

    // El servidor se queda bloqueado aquí esperando peticiones (.await)
    axum::Server::bind(&addr)
        .serve(app.into_make_service())
        .await
        .unwrap();
}

// Esta es la función que procesa los mensajes recibidos
async fn handle_log(Json(payload): Json<LogMessage>) -> &'static str {
    println!(
        "[{}] [{}] - {}",
        payload.source,
        payload.level.to_uppercase(),
        payload.message
    );
    "OK"
}
```

4. Orquestación con Docker Compose

Para garantizar la escalabilidad, se contenerización de los servicios. Se utilizaron Dockerfile de múltiples etapas usando “rust:1.83-slim” para lograr compilar los binarios

y copiándolos a imágenes muy ligeras basadas en “debian:bookworm-slim” con librerías libss13 instaladas.

Para ordenar la red, se implementó Docker Compose V2. En el archivo de “docker-compose.yml” de los workers, se declararon las variables del entorno necesarias para poder establecer su identidad y se limitaron sus recursos físicos de forma estricta a 128 MB de RAM y 0.25 CPUs para simular los entornos restringidos y forzar el trabajo distribuido.

```
version: '3.8'

services:
  # El servicio de logs debe ser el primero en estar disponible
  logger:
    build: .
    command: /app/logger
    ports:
      - "4000:4000" # Coincide con el puerto que definimos en el código
del logger
    mem_limit: 128m
    cpus: 0.25

  coordinator:
    build: .
    command: /app/hub
    ports:
      - "3000:3000"
    volumes:
      - ../output # Para guardar el archivo mandelbrot.png en tu máquina
    depends_on:
      - logger # Asegura que el logger inicie antes que el hub
    mem_limit: 256m
    cpus: 0.5

  api-gateway:
    build: .
    command: /app/api_gateway
    ports:
      - "3005:3005"
    depends_on:
```

```
- logger
mem_limit: 128m
cpus: 0.25

monitor:
  build: .
  command: /app/monitor
  ports:
    - "3002:3002"
  depends_on:
    - logger
  mem_limit: 128m
  cpus: 0.25
```

Para revisar a detalle la implementación completa, consultar la totalidad de los códigos fuente explicados anteriormente y acceder a los archivos de configuración de la infraestructura, se invita al lector a visitar el repositorio oficial del proyecto en GitHub: <https://github.com/Forajido24/Zyntra-Team/tree/main>

Conclusión

El desarrollo de esta práctica representó un reto, llevándonos a aplicar conocimientos de programación, virtualización y redes en un entorno que simuló los problemas del mundo real. Si bien el sistema de renderizado distribuido en Rust funcionó de manera excelente, la mayor parte de nuestra curva de aprendizaje y nuestras dificultades se concentraron en la capa de infraestructura y conectividad.

El obstáculo más demandante se presentó en las etapas iniciales al intentar establecer la VPN con WireGuard. Invertimos una cantidad considerable de tiempo tratando de diagnosticar por qué los nodos no lograban realizar el *handshake*. Tras una extensa investigación y consultas con compañeros y maestro, descubrimos que el problema radicaba en el CGNAT. Tres de los integrantes del equipo cuentan con el proveedor de internet Totalplay, el cual no asigna una dirección IPv4 pública de manera nativa, haciendo imposible que sus equipos actuarán como nodos accesibles desde el exterior.

Para solucionar esto, decidimos que el integrante que contaba con servicio de Telmex alojara el nodo coordinador. Pero, esto derivó en un nuevo desafío técnico debido a la arquitectura de Windows. Encontramos que el subsistema de Windows para Linux WSL2 tiene problemas propios para gestionar la VPN de manera local y enrutar el tráfico correctamente hacia los contenedores. Aunque se nos recomendó migrar el entorno a Hyper-V para tener un control de red, el proyecto ya se encontraba en una fase muy avanzada y tomar la decisión de empezar desde cero no era viable por los tiempos de entrega. En su lugar, optamos por una arquitectura híbrida y tuvimos que aprender a configurar las reglas del Firewall de Windows y la apertura de puertos específicos para permitir que WSL2 y WireGuard se comunicaran.

A las complejidades técnicas se sumaron los retos logísticos. Coordinar los tiempos de trabajo del equipo fue complicado, ya que nuestras disponibilidades de horario rara vez coincidían, lo que nos obligó a ser muy eficientes durante las ventanas de tiempo en las que podíamos conectarnos de forma síncrona para hacer pruebas de red integrales.

A pesar de estas frustraciones y del tiempo invertido en depurar conexiones en lugar de escribir código, la práctica resultó invaluable. Nos llevamos un aprendizaje profundo sobre cómo las restricciones de los proveedores de telecomunicaciones afectan los despliegues de software, la importancia de planificar la arquitectura de red desde el día uno, y la necesidad de tener bases sólidas en seguridad perimetral y firewalls. Fue un proyecto exhaustivo, pero nos dio experiencia práctica.

Referencias:

Microsoft. (2024). *Advanced settings configuration in WSL*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/wsl/wsl-config>

Docker Inc. (2024a). *Overview of Docker Compose*. Docker Documentation. <https://docs.docker.com/compose/>

Donenfeld, J. A. (2024). *WireGuard: Fast, modern, secure VPN tunnel*. WireGuard. <https://www.wireguard.com/>

Donenfeld, J. A. (2017). WireGuard: Next Generation Kernel Network Tunnel. *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

Weisstein, E. W. (2024). *Mandelbrot set*. MathWorld - A Wolfram Web Resource. <https://mathworld.wolfram.com/MandelbrotSet.html>

The Rust Project Developers. (2024). *The Rust programming language*. Rust Documentation. <https://doc.rust-lang.org/book/>

Rust Programming Tutorial. (s. f.). YouTube. <https://www.youtube.com/playlist?list=PLzMbGfZo4-nyLTISRBo0zjSnCnqjHYQ>